CWLog_Utils 閉域網向け ハンド修正指示書（メッセージ時刻検索修正）

■ 基本情報

対象システム / プロジェクト	CWLog_Utils (https://github.com/ProjectRubyRing/CWLog_Utils_Claude)
修正対象ファイル	lib/logs.sh （※ 本修正は 1 ファイルのみの変更で完結します）
作業種別	閉域網（オフライン環境）における手動コード改修（ハンド修正）
作成日時 / バージョン	2026-10-26 / v1.0.0
動作検証状況	構文チェック(bash -n)、時刻抽出精度、境界値内外、Excel出力フォーマット全項目検証済

■ 不具合の事象・原因・解決策の概要

発生している事象	start と end で日時範囲を指定してロググループの内容を Excel 出力した際、ログメッセージ列内の時刻ではなく、CloudWatch 内部の「@timestamp」(取込時刻・UTC) で範囲検索・抽出・出力が行われてしまう。
根本原因	1. lib/logs.sh 内 cw_fetch_via_insights のクエリで「sort @timestamp desc」が指定され、@message の範囲指定が欠落していた。 2. 取得結果のパース部で「ts = $1」として @timestamp (第1列) を抽出し、UTC として epoch ミリ秒変換していた。 3. その結果、Excel 出力(lib/xlsx.awk)の日時列 (B列) やソート順が @timestamp を基準として処理されていた。
修正方針 (最小変更)	lib/logs.sh の 3 箇所のみを改修： 【修正1】cw_fetch_via_insights のクエリに @message の範囲条件を追加し、sort @message desc に変更。 【修正2】パース部で「ts = $3」とし、@message (第3列) から時刻を抽出・ローカル時刻で epoch 変換、範囲外行をスキップ。 【修正3】cw_fetch_events にもメッセージ時刻での再評価フィルタを追加し、フィルタエンジン利用時も含めた完全防御を実現。

■ 修正箇所一覧

修正No	対象関数	修正前行番号	修正後行番号	修正区分	修正内容の概要	詳細シートリンク
1	cw_fetch_events	166行目 付近	166～188行目	追加 (23行)	イベント集約部でログメッセージ列 ($3) の時刻を抽出・範囲絞り込み・ソート	修正箇所詳細!A5
2	cw_fetch_via_insights	289行目 付近	312～316行目	置換 (1行→5行)	クエリを「sort @timestamp」から「filter @message >= start..end」+「sort @message」へ変更	修正箇所詳細!A18
3	cw_fetch_via_insights	346～368行目 付近	373～395行目	置換 (パース部)	パース元を $1(@timestamp) から $3(@message) に変更し、範囲外イベントを next で除外	修正箇所詳細!A31

■ 作業時の重要確認事項（閉域網端末向け）

1. 事前バックアップの徹底: 修正前に必ず `cp lib/logs.sh lib/logs.sh.bak` を実行してください。

2. 文字コード・改行コードの保持: UTF-8 / LF を維持してください（CRLF が混入しないよう注意）。

3. 構文チェックの実施: 編集完了後、必ず `bash -n lib/logs.sh` を実行して構文エラーがないことを確認してください。

4. 動作確認: `./cwlog --start 'YYYY-MM-DD HH:MM:SS' --end 'YYYY-MM-DD HH:MM:SS' --excel` を実行し、生成された Excel の B列（日時）がメッセージ内の時刻と一致することを確認してください。

修正箇所詳細 コード対比表（lib/logs.sh）

■ 修正 1: cw_fetch_events () への修正 [追加 (フィルタ処理の挿入)]

修正対象行（修正前）	166行目 付近	**vi検索キーワード**	/整列して上限件数
修正後行番号（目安）	166～188行目	**対象ファイル**	lib/logs.sh
解説・技術詳細	【目的と効果】 各エンジン(insights / filter)が取得した直後の $raw (TSV形式) に対し、メッセージ列 ($3) から正規表現で日付・時刻文字列を抽出し、ローカルタイムゾーンで epoch ミリ秒に変換して第1列 ($1) を上書きします。 さらに、引数の開始時刻 (start_ms) と終了時刻 (end_ms) の境界範囲外である行を「next」で除外します。 それにより、下位の filter エンジンを利用した場合や、Insights からの取得結果に対しても、最終的な整列（sort）および Excel 出力（xlsx_out）に適される日時がメッセージ列時刻と完全		

【修正前コード】(変更前 / 削除対象)	【修正後コード】(反映後 / 追加入力内容)

```
    ((rc != 0)) && return $rc

    # (時刻, ログストリーム) で整列して上限件数まで切り出す
    if [[ $order == asc ]]; then
        LC_ALL=C sort -t $'\t' -k1,1n -k2,2 "$raw" | head -n
"$limit" > "$outfile"
    else
        LC_ALL=C sort -t $'\t' -k1,1nr -k2,2r "$raw" | head -
n "$limit" > "$outfile"
    fi
```

```
    ((rc != 0)) && return $rc

    # ログメッセージ列内の時刻で再評価 & 範囲絞り込み
    local filtered="$CW_TMPDIR/events_filtered.tsv"
    awk -F'\t' -v OFS='\t' -v s="$start_ms" -v e="$end_ms" '
        {
            ts = $3
            sub(/^\[/, "", ts)
            gsub(/[-:\/T]/, " ", ts)
            gsub(/\./, " ", ts)
            n = split(ts, p, /[ ]+/)
            if (n >= 6) {
                epoch = mktime(sprintf("%s %s %s %s %s %s", p[1], p[2], p[3], p[4], p[5], p[6]))
                if (epoch > 0) {
                    ms = (n >= 7) ? p[7] + 0 : 0
                    $1 = epoch * 1000 + ms
                }
            }
            if (s > 0 && $1 < s) next
            if (e > 0 && $1 > e) next
            print
        }
    ' "$raw" > "$filtered"
    mv "$filtered" "$raw"

    # (時刻, ログストリーム) で整列して上限件数まで切り出す
    if [[ $order == asc ]]; then
        LC_ALL=C sort -t $'\t' -k1,1n -k2,2 "$raw" | head -n "$limit" > "$outfile"
    else
        LC_ALL=C sort -t $'\t' -k1,1nr -k2,2r "$raw" | head -n "$limit" > "$outfile"
    fi
```

■ 修正 2: cw_fetch_via_insights () への修正 [置換 (クエリ文字列構築)]

修正対象行（修正前）	289行目 付近	**vi検索キーワード**	`/sort @timestamp`
修正後行番号（目安）	312〜316行目	**対象ファイル**	lib/logs.sh

解説・技術詳細	【目的と効果】 CloudWatch Logs Insights のクエリ文字列において、取込時刻 @timestamp によるサーバ側ソート指定 (sort @timestamp desc) を削除し、メッセージ列 (@message) に対する日時範囲フィルタを追加した上で、メッセージ列での降順ソート (sort @message desc) に変更します。 通常形式 (YYYY-MM-DD HH:MM:SS) および角括弧形式 ([YYYY-MM-DD ...) の両方に対応する論理式 (OR) を設定することで、AWS サーバ側でメッセージ列の時刻による絞り込みが正確に機能します。

【修正前コード】(変更前 / 削除対象)	【修正後コード】(反映後 / 追加入力内容)

`if [[ -n $pattern ]]; then`
`  local escaped=${pattern//\\/\\\\}`
`  escaped=${escaped//\"/\\\"}`
`  query+=" | filter @message like \"$escaped\""`
`fi`
`query+=" | sort @timestamp desc | limit $query_limit"`
	`if [[ -n $pattern ]]; then`
	`  local escaped=${pattern//\\/\\\\}`
	`  escaped=${escaped//\"/\\\"}`
	`  query+=" | filter @message like \"$escaped\""`
	`fi`
	`local start_str end_str`
	`start_str=$(cw_tzrun date -d "@$((start_ms / 1000))" '+%Y-%m-%d %H:%M:%S')`
	`end_str=$(cw_tzrun date -d "@$((end_ms / 1000))" '+%Y-%m-%d %H:%M:%S')`
	`query+=" | filter (@message >= \"$start_str\" and @message <= \"$end_str\") or (@message >= \"[$start_str\" and @message <= \"[$end_str\")"`
	`query+=" | sort @message desc | limit $query_limit"`

■ 修正 3: cw_fetch_via_insights () への修正 [置換 (結果TSVパース部)]

修正対象行（修正前）	346～368行目 付近	**vi検索キーワード**	`/ts = $1`
修正後行番号（目安）	373～395行目	**対象ファイル**	lib/logs.sh
解説・技術詳細	【目的と効果】 Logs Insights の queryId 結果から jq で抽出した TSV を処理する awk スクリプトを修正します。 従来: $1 (@timestamp) を抽出し、mktime(..., 1) で UTC として解釈して $1 に再代入。 修正後: $3 (@message) から時刻文字列を抽出。先頭の '[' 除去や '/', '-', ':', 'T' 区切りを空白化して split。		

【修正前コード】(変更前 / 削除対象)	【修正後コード】(反映後 / 追加入力内容)

```
    # @timestamp は 'YYYY-MM-DD HH:MM:SS.mmm' (UTC)
    jq -r '
        .results[]?
        | ( reduce .[] as $f ({}; .[$f.field] = $f.value) )
        | [ (.["@timestamp"] // ""), (.["@logStream"] // ""),
((.["@message"] // "") | sub("\n+$"; "")) ]
        | @tsv
    ' "$out" \
        | awk -F'\t' -v OFS='\t' '
            {
                ts = $1
                gsub(/[-:]/, " ", ts)
                gsub(/\./, " ", ts)
                n = split(ts, p, /[ ]+/)
                if (n >= 6) {
                    epoch = mktime(sprintf("%s %s %s %s %s
%s", p[1], p[2], p[3], p[4], p[5], p[6]), 1)
                    ms = (n >= 7) ? p[7] + 0 : 0
                    $1 = (epoch < 0) ? 0 : (epoch * 1000 +
ms)
                } else {
                    $1 = 0
                }
                print
            }
        ' >> "$raw"
```

```
    # ログメッセージ列 (@message) 内の時刻を抽出して epoch ミリ秒へ変換
    jq -r '
        .results[]?
        | ( reduce .[] as $f ({}; .[$f.field] = $f.value) )
        | [ (.["@timestamp"] // ""), (.["@logStream"] // ""), ((.["@message"] // "") | sub("\n+$";
"")) ]
        | @tsv
    ' "$out" \
        | awk -F'\t' -v OFS='\t' -v s="$start_ms" -v e="$end_ms" '
            {
                ts = $3
                sub(/^\[/, "", ts)
                gsub(/[-:\/T]/, " ", ts)
                gsub(/\./, " ", ts)
                n = split(ts, p, /[ ]+/)
                if (n >= 6) {
                    epoch = mktime(sprintf("%s %s %s %s %s %s", p[1], p[2], p[3], p[4], p[5],
p[6]))
                    ms = (n >= 7) ? p[7] + 0 : 0
                    $1 = (epoch < 0) ? 0 : (epoch * 1000 + ms)
                }
                if (s > 0 && $1 < s) next
                if (e > 0 && $1 > e) next
                print
            }
        ' >> "$raw"
```

閉域網端末における vi エディタ手動修正手順書

手順	フェーズ	操作対象	実行コマンド / viキー操作	入力・貼付コード / 編集内容	操作後の確認ポイント
1	事前準備	**バックアップ**	`cp lib/logs.sh lib/logs.sh.bak_$(date +%Y%m%d_%H%M%S)`	（バックアップファイルの作成） 元の lib/logs.sh が破損した場合にいつでも即座に復元できるようにバックアップを取得します。	ls -l lib/logs.sh* でバックアップファイルが正常作成されていることを確認。
2	ファイルオープン	**lib/logs.sh**	`vi lib/logs.sh`	（vi で対象ファイルを開く） 行番号を表示したい場合は vi 内で `:set number` を実行します。	画面左下に対象ファイル名が表示され、閲覧モードになっていることを確認。

3	修正箇所1	**cw_fetch_events
		(166行目 付近)**
			**/整列して上限件数
			<Enter>
			O（大文字O：上行挿入）**
				【以下の22行を挿入】
				`# ログメッセージ列内の時刻で再評価 & 範囲絞り込み`
				`local filtered="$CW_TMPDIR/events_filtered.tsv"`
				`awk -F'\t' -v OFS='\t' -v s="$start_ms" -v e="$end_ms" '`
				`{`
				`ts = $3`
				`sub(/^\[/, "", ts)`
				`gsub(/[-:\/T]/, " ", ts)`
				`gsub(/\./, " ", ts)`
				`n = split(ts, p, /[ ]+/)`
				`if (n >= 6) {`
				`epoch = mktime(sprintf("%s %s %s %s %s %s", p[1], p[2], p[3], p[4], p[5], p[6]))`
				`if (epoch > 0) {`
				`ms = (n >= 7) ? p[7] + 0 : 0`
				`$1 = epoch * 1000 + ms`
				`}`
				`}`
				`if (s > 0 && $1 < s) next`
				`if (e > 0 && $1 > e) next`
				`print`
				`}`
				`' "$raw" > "$filtered"`
				`mv "$filtered" "$raw"`
				
				入力完了後、<Esc> キーを押してノーマルモードに戻ります。
					「((rc != 0)) && return $rc」と「# (時刻, ログストリーム) で整列」の間に上記ブロックが正しく配置されたことを確認。

4	修正箇所2	**cw_fetch_via_insights
		(クエリ構築部)**
			**/sort @timestamp
			<Enter>
			dd（1行削除）
			O（大文字O：上行挿入）**
				【以下の5行を入力】
				`local start_str end_str`
				`start_str=$(cw_tzrun date -d "@$((start_ms / 1000))" '+%Y-%m-%d %H:%M:%S')`
				`end_str=$(cw_tzrun date -d "@$((end_ms / 1000))" '+%Y-%m-%d %H:%M:%S')`
				`query+=" | filter (@message >= \"$start_str\" and @message <= \"$end_str\") or (@message >= \"[$start_str\" and @message <= \"[$end_str\")"`
				`query+=" | sort @message desc | limit $query_limit"`
					「query+=" | sort @timestamp desc | limit $query_limit"」が削除され、上記の5行に置き換わっていることを確認。

5	修正箇所3	**cw_fetch_via_insights (結果パース部)**	**/ts = \$1 <Enter>**	【awk 処理部分の修正】 1. awk 引数変更: `awk -F'\t' -v OFS='\t'` を `awk -F'\t' -v OFS='\t' -v s="$start_ms" -v e="$end_ms"` に書き換える。 2. 日時抽出元変更: `ts = $1` の「1」を「3」に変更して `ts = $3` にする。 3. 前処理追加: `ts = $3` の下に以下3行を挿入: sub(/^\[/, "", ts) gsub(/[-:\/T]/, " ", ts) gsub(/\./, " ", ts) 4. mktime引数変更: `mktime(..., 1)` 末尾の「, 1」を削除してローカル時刻解釈にする。 5. 範囲外行スキップ追加: `print` の直前に以下2行を挿入: if (s > 0 && $1 < s) next if (e > 0 && $1 > e) next 入力完了後、<Esc> キーを押してノーマルモードに戻ります。	awk ブロック内で $3 から時刻が抽出され、s/e による範囲判定が入っていることを確認。
6	保存終了	**vi終了**	**:wq <Enter>**	編集内容を保存して vi を終了します。	シェルプロンプトに戻り、エラーメッセージ等が出ていないことを確認。
7	構文チェック	**bash構文検証**	**bash -n lib/logs.sh**	（スクリプトを実行せずに構文エラー・クォート閉じ忘れ等がないか検査）	何も出力されずリターンコード 0 ($? == 0) であれば構文正常。
8	差分確認	**git差分確認**	**git diff lib/logs.sh**	（修正内容が意図通りか diff で最終目視確認）	「Unified_Diff」シートの差分と完全に一致していることを確認。

lib/logs.sh 修正差分 (git diff -u 形式)

行	区分	Diff コード内容
1		diff --git a/lib/logs.sh b/lib/logs.sh
2		--- a/lib/logs.sh
3		+++ b/lib/logs.sh
4	@@	@@ -163,6 +163,29 @@ cw_fetch_events() {
5		fi
6		((rc != 0)) && return $rc
7		
8	+	+ # ログメッセージ列内の時刻で再評価 & 範囲絞り込み
9	+	+ local filtered="$CW_TMPDIR/events_filtered.tsv"
10	+	+ awk -F'\t' -v OFS='\t' -v s="$start_ms" -v e="$end_ms" '
11	+	+ {
12	+	+ ts = $3
13	+	+ sub(/^\[/, "", ts)
14	+	+ gsub(/[-:\/T]/, " ", ts)
15	+	+ gsub(/\./, " ", ts)
16	+	+ n = split(ts, p, /[ ]+/)
17	+	+ if (n >= 6) {
18	+	+ epoch = mktime(sprintf("%s %s %s %s %s %s", p[1], p[2], p[3], p[4], p[5], p[6]))
19	+	+ if (epoch > 0) {
20	+	+ ms = (n >= 7) ? p[7] + 0 : 0
21	+	+ $1 = epoch * 1000 + ms
22	+	+ }
23	+	+ }
24	+	+ if (s > 0 && $1 < s) next
25	+	+ if (e > 0 && $1 > e) next

```
26  +  +            print
27  +  +        }
28  +  +    ' "$raw" > "$filtered"
29  +  +    mv "$filtered" "$raw"
30  +  +
31     # (時刻, ログストリーム) で整列して上限件数まで切り出す
32     if [[ $order == asc ]]; then
33         LC_ALL=C sort -t $'\t' -k1,1n -k2,2 "$raw" | head -n "$limit" > "$outfile"
34  @@ @@ -286,7 +309,11 @@ cw_fetch_via_insights() {
35         escaped=${escaped//\"/\\\"}
36         query+=" | filter @message like \"$escaped\""
37     fi
38  -  -    query+=" | sort @timestamp desc | limit $query_limit"
39  +  +    local start_str end_str
40  +  +    start_str=$(cw_tzrun date -d "@$((start_ms / 1000))" '+%Y-%m-%d %H:%M:%S')
41  +  +    end_str=$(cw_tzrun date -d "@$((end_ms / 1000))" '+%Y-%m-%d %H:%M:%S')
42  +  +    query+=" | filter (@message >= \"$start_str\" and @message <= \"$end_str\") or (@message >= \"[$start_str\" and @message <= \"[$end_str\")"
43  +  +    query+=" | sort @message desc | limit $query_limit"
44
45     local rc=0
46     cw_aws_run "$out" logs start-query \
47  @@ @@ -343,26 +370,27 @@ cw_fetch_via_insights() {
48         return 1
49     fi
50
51  -  -    # @timestamp は 'YYYY-MM-DD HH:MM:SS.mmm' (UTC)
52  +  +    # ログメッセージ列 (@message) 内の時刻を抽出して epoch ミリ秒へ変換
53     jq -r '
54         .results[]?
```

```
55      | ( reduce .[] as $f ({}; .[$f.field] = $f.value) )
56      | [ (.["@timestamp"] // ""), (.["@logStream"] // ""), ((.["@message"] // "") | sub("\n+$"; "")) ]
57      | @tsv
58  ' "$out" \
59  -   - | awk -F'\t' -v OFS='\t' '
60  +   + | awk -F'\t' -v OFS='\t' -v s="$start_ms" -v e="$end_ms" '
61          {
62  -   -           ts = $1
63  -   -           gsub(/[-:]/, " ", ts)
64  +   +           ts = $3
65  +   +           sub(/^\[/, "", ts)
66  +   +           gsub(/[-:\/T]/, " ", ts)
67              gsub(/\./, " ", ts)
68              n = split(ts, p, /[ ]+/)
69              if (n >= 6) {
70  -   -               epoch = mktime(sprintf("%s %s %s %s %s %s", p[1], p[2], p[3], p[4], p[5], p[6]), 1)
71  +   +               epoch = mktime(sprintf("%s %s %s %s %s %s", p[1], p[2], p[3], p[4], p[5], p[6]))
72                  ms = (n >= 7) ? p[7] + 0 : 0
73                  $1 = (epoch < 0) ? 0 : (epoch * 1000 + ms)
74  -   -           } else {
75  -   -               $1 = 0
76              }
77  +   +           if (s > 0 && $1 < s) next
78  +   +           if (e > 0 && $1 > e) next
79              print
80          }
81      ' >> "$raw"
```